Contenido

Pregunta 2: Características de una relación de herencia 2
- Relación "ES-UN" (IS-A): 2
- Herencia de atributos y métodos: 2
- Reutilización de código: 2
- Especialización: 2
- Sobrescritura (Override): 2
- Jerarquía de clases: 2
- Polimorfismo: 2

Pregunta 3: Herencia múltiple 3

¿Cómo se realiza la herencia múltiple? 3

Problemas de la herencia múltiple: 3

Problema del diamante (Diamond Problem): 3

Ambigüedad en la resolución de métodos: 4

Complejidad en el diseño: 4

Conflictos de nombres: 4

Soluciones en diferentes lenguajes: 4

Pregunta 2: Características de una relación de herencia

Para que una relación entre dos clases pueda ser calificada como **herencia**, debe cumplir las siguientes características:

- ## Relación "ES-UN" (IS-A):

  La clase hija debe ser un tipo especializado de la clase padre. Por ejemplo, un Coche "ES-UN" Vehículo.

- ## Herencia de atributos y métodos:

  La clase hija hereda automáticamente todos los atributos y métodos (públicos y protegidos) de la clase padre.

- ## Reutilización de código:

  Permite reutilizar el código de la clase padre sin necesidad de duplicarlo.

- ## Especialización:

  La clase hija puede añadir nuevos atributos y métodos específicos, además de los heredados.

- ## Sobrescritura (Override):

  La clase hija puede modificar el comportamiento de los métodos heredados proporcionando su propia implementación.

- ## Jerarquía de clases:

  Se establece una relación jerárquica donde la clase padre representa conceptos más generales y la clase hija conceptos más específicos.

- ## Polimorfismo:

  Permite tratar objetos de la clase hija como si fueran objetos de la clase padre.

Pregunta 3: Herencia múltiple

¿Cómo se realiza la herencia múltiple?

La **herencia múltiple** permite que una clase derive de más de una clase base, heredando atributos y métodos de todas ellas.

Ejemplo conceptual (en lenguajes que lo soportan como C++):

```
// Ejemplo en C++ (no válido en Java)
class Vehiculo {
   // atributos y métodos de vehículo
};

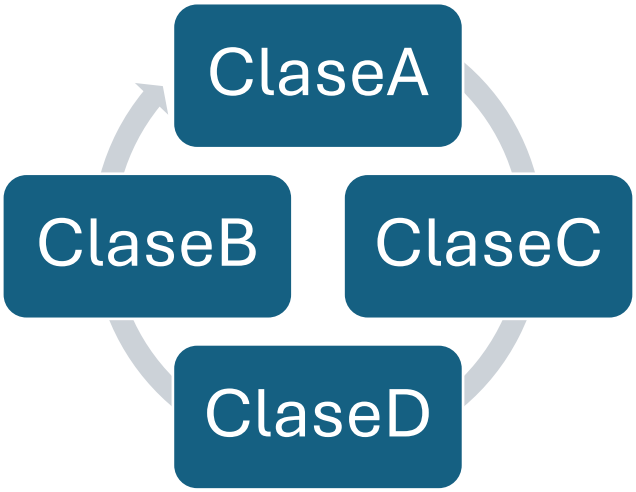
class VehiculoElectrico {
   // atributos y métodos específicos de vehículos eléctricos
};

// CocheEléctrico hereda de ambas clases
class CocheElectrico : public Vehiculo, public VehiculoElectrico {
   // Hereda de ambas clases
};
```

Problemas de la herencia múltiple:

Problema del diamante (Diamond Problem):

- Ocurre cuando dos clases base heredan de una misma clase abuela, y una clase hija hereda de ambas clases base.
- La clase hija tiene dos copias de los atributos/métodos de la clase abuela, creando ambigüedad.



Ambigüedad en la resolución de métodos:

- Si ambas clases padre tienen un método con el mismo nombre, el compilador no sabe cuál invocar.

Complejidad en el diseño:

- Hace que la jerarquía de clases sea más compleja y difícil de mantener.

Conflictos de nombres:

- Pueden surgir conflictos cuando clases padre tienen atributos o métodos con el mismo nombre.

Soluciones en diferentes lenguajes:

Java:

No permite herencia múltiple de clases, pero sí de interfaces.

C++:

Permite herencia múltiple pero usa clases virtuales para resolver el problema del diamante.

Python:

Permite herencia múltiple y usa el MRO (Method Resolution Order) para resolver conflictos.